

get_eth

REFERENCE

API consumer

read the reader's wired Ethernet interface configuration and status

Overview

`get_eth` retrieves the reader's **wired Ethernet interface configuration and status** — interface state, live link status and speed, and the DHCP-assigned IPv4 addressing. It is a read-only query on the device-management ([MGMT](#)) endpoint, useful for verifying that the reader's wired uplink is up and reaching back-end services.

The response shape depends on interface state: when Ethernet is **enabled** the device returns the full `interfaceDetails` (hostname, MAC, link, IPv4); when it is **disabled** it returns only `interfaceName` and `status` (still a `code 0` success). Note that, per the command's own contract, **static IP configuration and IPv6 are not available in the current API version**, and the 802.1X `security` block is not populated (see [Response Semantics](#) below).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_wifi</code> , <code>get_status</code>

When to Use

Use `get_eth` when you need to **verify the reader's wired network state** — during commissioning, provisioning verification, or troubleshooting a wired uplink. Because it is read-only and idempotent, it is safe to poll.

- **Confirm connectivity:** check `interfaceDetails.status` (enabled?), `linkStatus.status` (cable connected?), and `linkStatus.linkSpeed` (negotiated speed).
- **Validate addressing:** check `ipv4Configuration.ipAddress`, `gateway`, and `enableDhcp` to confirm the reader has a working IPv4 route to the broker / back-end.
- **Troubleshoot wired issues:** an unexpected `linkSpeed`, a missing IP, or a `Disabled` interface each point to a specific cable/switch/config problem.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `ethConfig.interfaceDetails.status` (response) — `Enabled` or `Disabled`. When `Disabled`, only `interfaceName` and `status` are returned.
- `ethConfig.interfaceDetails.linkStatus` (response) — `status` (`Connected` / `Disconnected`) and `linkSpeed`; the live carrier state.

- `ethConfig.interfaceDetails.ipv4Configuration` (response) — the DHCP-assigned IPv4 addressing (reported read-only).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_eth" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command*	string	—	get_eth	Fixed command name. Retrieves the Ethernet configuration.
2	requestId*	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and an `ethConfig` object. On the **Schema** tab, expand `ethConfig` → `interfaceDetails` (and its `linkStatus`, `ipv4Configuration`, `security` sub-objects) to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full (enabled-interface) JSON. Within `interfaceDetails`, only `interfaceName` and `status` are required; the rest are returned when the interface is **enabled**. The `security` block is reserved (not available in the current API version), and `ipv4Configuration` is reported read-only.

Interface state

Ethernet config info

Response payload

Example: Interface enabled — full config

```
1  {
2    "command" : "get_eth" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "ethConfig" : {
6      "interfaceDetails" : {
7        "interfaceName" : "eth0" ,
8        "status" : "Enabled" ,
9        "hostname" : "RFD40-211545201D0020" ,
10       "macAddress" : "AC:3F:A4:00:2C:89" ,
11       "linkStatus" : {
12         "status" : "Connected" ,
13         "linkSpeed" : "100Mbps"
14       } ,
15       "ipv4Configuration" : {
16         "ipAddress" : "192.168.0.101" ,
17         "subnetMask" : "255.255.255.0" ,
18         "gateway" : "192.168.0.1" ,
19         "enableDhcp" : true ,
20         "dnsServer" : "0.0.0.0" ,
21         "domainName" : "zebra.com"
22       }
23     } ,
24   } ,
25   "response" : {
26     "code" : 0 ,
27     "description" : "Success"
28   }
29 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_eth	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command; emitted per-handler, not device-wide.
4	ethConfig	object	—	—	Ethernet configuration container. Present on a successful read; optional at the envelope level.
5	interface Details	object	—	—	Details of the Ethernet interface. <code>interfaceName</code> and <code>status</code> are always present; the rest are present only when the interface is enabled.
6	inter fac eNam e *	string	—	eth0	OS-level interface name.
7	stat us *	enum	Enabled Disabled	enabled	Interface enable state. When Disabled, the device returns only <code>interfaceName</code> and <code>status</code> .
8	hostna me	string	—	RFD40AB03	Hostname bound to the interface (present when enabled).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
9	macAddress	string	—	E0-D0-45-3D-38-1D	MAC address of the interface (present when enabled).
10	linkStatus	object	—	—	Live carrier state and negotiated speed (present when enabled).
11	status *	enum	Connected Disconnected	Connected	Physical link state (required within linkStatus).
12	linkSpeed *	string	—	100Mbps	Negotiated link speed (required within linkStatus).
13	ipv4Configuration	object	—	—	IPv4 addressing (present when enabled). Reported read-only — DHCP-assigned; static IP is not settable via this API (see Response Semantics).
14	ipAddress	string	ipv4	192.168.1.81	IPv4 address of the interface.
15	subnetMask	string	ipv4	255.255.255.0	Subnet mask.
16	gateway	string	ipv4	192.168.1.144	Default gateway.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
17	enableDhcp	boolean	—	true	Whether DHCP is enabled for IPv4 assignment.
18	dnsServer	string	ipv4	192.168.1.78	DNS server address.
19	domainName	string	—	example.com	Domain name.
20	security	object	—	—	802.1X security block. Reserved — not available in the current API version (planned for a future release); not populated. See Response Semantics.
21	securityStatus	boolean	—	true	Whether 802.1X security is enabled.
22	EAP_802_1X	object	—	—	802.1X EAP parameters (reserved).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	authentication	enum	EAP-MD5 LEAP EAP-SIM EAP-AKA EAP-TLS EAP-TTLS EAP-FAST	PEAP	Outer EAP authentication method.
24	inner authentication	enum	EAP-MS-CHAPv2 EAP-TLS EAP-GTC	EAP-MS-CHAPv2	Inner authentication method.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
25	certificate	string	—	client45	Certificate name used for EAP-TLS.
26	response *	object	—	—	Result wrapper.
27	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
28	description *	string	—	Success	Human-readable status.

Response Semantics

- **Not available in the current API version.** Per this command's contract, **static IP configuration and IPv6 are not available in the current API version.** The response schema models a `security` / `EAP802_1X` block and a full `ipv4Configuration`, but:
 - the `security` block is **reserved** (planned for a future release) and is **not populated**;
 - `ipv4Configuration` is **reported read-only** — the reader's addressing is DHCP-assigned, and you **cannot set a static IP** through this API;
 - **no IPv6** addressing is returned.
- **Disabled interface.** When Ethernet is disabled, the response contains only `ethConfig.interfaceDetails.interfaceName` and `status (Disabled)` — `linkStatus`, `ipv4Configuration`, `hostname`, and `macAddress` are absent. This is normal behavior and a `code 0` success, not an error.

Response Codes

`get_eth` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — Ethernet configuration retrieved and returned. A disabled interface is also a 0 success (minimal payload), not an error.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the interface details at this time. Retry after a short delay; if it persists, reboot the device.
101	Default error — generic catch-all.

Examples

`get_eth` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The **Interface state** dropdown under [Response Payload](#) shows both replies: the full reply for an enabled interface, and the minimal one for a disabled interface, which carries only `interfaceName` and `status` and is still a `code 0` success.

Notes and Caveats

`get_eth` is a read-only getter — safe to poll during commissioning or troubleshooting. For the concept-level view of wired networking, see [Network Configuration](#); for Wi-Fi, use `get_wifi`.

Related:  [Network Configuration](#) ·  `get_wifi` ·  `get_status` ·  Network and Connectivity ·  [Response Codes](#)